E-Commerce Backend

Technical Documentation

Version 1.0

February 7, 2026

Table of Contents

1. Requirement Description

Overview

The e-commerce backend system is designed to provide a comprehensive, scalable, and maintainable API for managing product data from multiple sources. The system emphasizes clean architecture principles, modularity, and cloud-readiness.

Core Requirements

- Build a modular e-commerce backend API that supports product ingestion through web scraping from multiple sources
- Implement data normalization and categorization capabilities to ensure consistent product information
- Expose data via RESTful JSON API endpoints with comprehensive filtering and search capabilities
- Provide analytics endpoints for administrative users to monitor system performance and product metrics
- Maintain clean, testable, and extensible architecture with clear separation of concerns
- Support background job processing for scraping tasks and data aggregation
- Ensure cloud-readiness with AWS integration capabilities for storage, messaging, and observability

Functional Requirements

Product Management

- CRUD operations for products with full attribute support
- Product categorization using rule-based and machine learning approaches
- Advanced filtering by category, price range, attributes, and source
- Full-text search capabilities

Data Ingestion

- Web scraping from multiple e-commerce sources (Amazon, eBay, etc.)
- Data normalization to standardize product information across sources
- Scheduled scraping jobs with status tracking
- Error handling and retry mechanisms

Analytics & Reporting

- Sales summary reports
- Top products analysis
- Category distribution metrics
- Dashboard visualizations for administrative users

Non-Functional Requirements

- Scalability: System must handle increasing data volumes and concurrent requests
- Maintainability: Code must be well-documented, modular, and follow best practices
- Testability: All components must be independently testable with high code coverage
- Performance: API responses should be optimized with caching where appropriate
- Security: Implement authentication, authorization, and data validation
- Observability: Comprehensive logging and monitoring capabilities

2. Architecture Description & ABB Matrix

Architecture Overview

The system follows Clean Architecture principles with four distinct layers, ensuring separation of concerns and maintaining independence from frameworks and external systems. This architecture promotes testability, maintainability, and flexibility.

Architectural Layers

1. Presentation Layer

Handles HTTP requests and responses, input validation, and serialization. Built using FastAPI framework with Pydantic schemas for request/response validation.

- REST API endpoints for all resources
- Request/response serialization using Pydantic models
- Input validation and error handling
- Authentication and authorization middleware

2. Application Layer

Orchestrates business workflows and use cases. Contains service interfaces and their implementations that coordinate domain models and infrastructure components.

- Business logic orchestration
- Transaction management
- Service interfaces and implementations
- Use case coordination

3. Domain Layer

Contains core business entities and domain logic. Framework-agnostic and represents the heart of the business rules. All business invariants are enforced here.

- Core business entities (Product, User, Order, Category)
- Business rules and invariants
- Domain interfaces and contracts
- Value objects and enumerations

4. Infrastructure Layer

Implements external concerns such as database access, caching, message queues, and external service integrations. Provides concrete implementations of domain interfaces.

- Database repositories and ORM mappings
- Cache implementations (Redis)
- Message queue integrations
- External API clients

ABB Matrix (Architecture Building Blocks)

The following table defines the major architectural building blocks and their responsibilities:

ABB	Responsibility	Key Modules	Interfaces	Notes
Presentation Layer	REST/JSON API and request/response validation	app/api	API routes, serializers	FastAPI routes, Pydantic schemas
Application Layer	Orchestrate use cases	app/core/services	Service interfaces	Business workflows
Domain Layer	Core entities and rules	app/core/models	Domain models	OOP models, dataclasses
Infrastructure Layer	Persistence, cache, queues	app/db, app/utils	Repository interfaces	Replace in-memory repos with DB

3. Components Description & SBB Matrix

Components Overview

Components are solution building blocks (SBBs) that encapsulate specific capabilities within the system. Each component is designed to be independently testable, maintainable, and replaceable. Components expose well-defined interfaces to minimize coupling and maximize cohesion.

Design Principles

- Single Responsibility: Each component handles one specific business capability
- Interface Segregation: Components expose minimal, focused interfaces
- Dependency Inversion: Components depend on abstractions, not concrete implementations
- Independent Testability: Each component can be tested in isolation

SBB Matrix (Solution Building Blocks)

The following table defines all major solution building blocks:

SBB	Responsibility	Key Modules	Interfaces
API Gateway	Serve REST endpoints	app/api/routes	Route handlers
Auth Service	Login/authentication	app/core/services/auth_service.py	AuthServiceInterface
Product Service	Product CRUD and filters	app/core/services/product_service.py	ProductServiceInterfa
Order Service	Order workflows	app/core/services/order_service.py	OrderServiceInterface
Scraping Service	Fetch and normalize products	app/scraper	Scraper base
Categorization Engine	Classify products	app/categorization	RuleBasedCategorize
Filtering Engine	Apply filters to products	app/categorization/filters	FilterEngine
Analytics Engine	Aggregate metrics	app/analytics	AnalyticsService
Notification Service	Future extension for notifications	N/A	N/A

4. Root Structure & Modules Explained

Project Structure

The project follows a modular structure that aligns with Clean Architecture principles. Each directory has a specific purpose and contains related functionality.

Directory Structure

app/api

Contains all presentation layer components including REST endpoint definitions, request/response serializers, and dependency injection setup.

- routes/: API endpoint definitions organized by resource
- schemas/: Pydantic models for request/response validation
- dependencies.py: Dependency injection configuration
- middleware.py: Custom middleware for authentication, logging, etc.

app/core

Houses the domain and application layers, containing business logic, domain models, and service interfaces.

- models/: Domain entity definitions (User, Product, Order, Category)
- services/: Business logic orchestration and service interfaces
- interfaces/: Abstract base classes and protocols
- exceptions.py: Custom exception definitions

app/scraper

Implements the web scraping functionality including base classes, source-specific scrapers, and job scheduling.

- base.py: Abstract Scraper base class defining the scraping interface
- sources/: Concrete scraper implementations (AmazonScraper, EbayScraper)
- scheduler.py: Background job scheduling for scraping tasks
- normalizer.py: Data normalization utilities

app/categorization

Provides product categorization capabilities using both rule-based and machine learning approaches, plus advanced filtering.

- rule_based.py: Rule-based categorization engine
- ml_based.py: Machine learning categorization models
- filters/: Product filtering engine and filter implementations
- rules.json: Categorization rules configuration

app/analytics

Analytics and reporting capabilities including metrics aggregation and dashboard views.

- service.py: Analytics service for computing metrics
- dashboard.py: Dashboard view renderers
- aggregators.py: Data aggregation utilities
- metrics.py: Metric calculation functions

app/db

Infrastructure layer components for data persistence, including repository implementations and database migrations.

- repositories/: Concrete repository implementations
- migrations/: Database schema migrations
- connection.py: Database connection management
- session.py: Session factory and transaction management

app/utils

Shared utility functions and helpers used across the application.

- id_generator.py: Unique ID generation utilities
- time_utils.py: Date/time manipulation helpers
- validators.py: Common validation functions
- logger.py: Logging configuration

docs

Project documentation including architecture decisions, API specifications, and roadmap.

- architecture.md: Detailed architecture documentation
- api_spec.yaml: OpenAPI specification
- roadmap.md: Development roadmap and future features

5. Classes & Objects Defined

Core Domain Models

The following classes represent the core business entities in the system. They are framework-agnostic and contain the essential business rules.

User

Represents a system user with authentication and authorization capabilities.

- id: Unique identifier for the user
- email: User's email address (unique)
- role: User role (admin, customer, etc.)
- is_active: Boolean flag indicating if the account is active

Product

Represents a product in the catalog with all associated metadata.

- id: Unique identifier for the product
- name: Product name/title
- description: Detailed product description
- price: Current price (decimal)
- category_id: Foreign key reference to Category
- attributes: Dictionary of product-specific attributes
- source: Origin of the product data (Amazon, eBay, etc.)
- created_at: Timestamp of product creation

Category

Represents a product category with support for hierarchical structures.

- id: Unique identifier for the category
- name: Category name
- parent_id: Foreign key to parent category (null for root categories)

Order

Represents a customer order containing one or more products.

- id: Unique identifier for the order
- user_id: Foreign key reference to User
- items: List of OrderItem objects
- total_price: Total order amount (calculated from items)
- status: Order status (pending, completed, cancelled, etc.)

OrderItem

Represents an individual item within an order.

- product_id: Foreign key reference to Product

- quantity: Number of units ordered
- unit_price: Price per unit at time of order

Scraping Layer Classes

Scraper (Abstract Base Class)

Defines the interface for all scraper implementations. All concrete scrapers must implement these methods.

- fetch(): Retrieves raw HTML/data from the source
- parse(raw): Extracts structured data from raw content
- normalize(parsed): Standardizes parsed data to common format

AmazonScraper

Concrete implementation of Scraper for Amazon product data.

- Inherits from Scraper base class
- Implements Amazon-specific parsing logic
- Handles Amazon-specific data structures and formats

EbayScraper

Concrete implementation of Scraper for eBay product data.

- Inherits from Scraper base class
- Implements eBay-specific parsing logic
- Handles eBay-specific data structures and formats

ScrapeJob

Represents a scheduled or executed scraping job with status tracking.

- source: The data source being scraped (Amazon, eBay, etc.)
- status: Current job status (pending, running, completed, failed)
- last_run: Timestamp of last execution

Categorization & Filtering Classes

RuleBasedCategorizer

Implements rule-based product categorization using keyword matching and pattern recognition.

- classify(product_name): Determines category based on product name using predefined rules
- Uses configurable rule sets from rules.json
- Returns category ID or None if no match

MLBasedCategorizer

Implements machine learning-based product categorization using trained models.

- classify(product_name): Uses ML model to predict category
- Supports model training and retraining
- Returns category ID with confidence score

FilterEngine

Applies complex filtering logic to product collections.

- apply_filters(products, criteria): Filters product list based on provided criteria
- Supports multiple filter types: category, price range, attributes, source
- Chainable filters for complex queries

Analytics & Dashboard Classes

AnalyticsService

Provides analytics and reporting capabilities for business metrics.

- sales_summary(): Generates sales summary report with totals and trends
- top_products(): Returns list of top-selling products
- category_distribution(): Analyzes product distribution across categories

DashboardView

Renders analytical metrics for administrative dashboard.

- render_metrics(): Formats and presents analytics data for dashboard display
- Supports multiple visualization formats
- Configurable metric selection and time ranges

Conclusion

This documentation provides a comprehensive overview of the e-commerce backend system architecture, components, and implementation details. The system is designed with modularity, testability, and scalability as core principles, following Clean Architecture patterns to ensure long-term maintainability and extensibility.

Next Steps

- Implement database repositories replacing in-memory storage
- Add caching layer with Redis
- Integrate message queue for background jobs
- Implement comprehensive test suite
- Set up CI/CD pipeline
- Deploy to AWS infrastructure